# WEEKLY INTERNSHIP REPORT

## Week 2

| | |
|---|---|
| **Full Name** | Muhammad Abdullah |
| **Internship Domain** | Frontend Development |
| **Email Address** | *m.abdullah98367@gmail.com* |
| **Phone Number** | *03062115084* |

# 1. Overview

During Week 2 of the internship, the Neon UI project from Week 1 was extended to fetch and display live data from a public API. The goal was to move beyond static, hardcoded content and connect the application to a real internet data source, handling the full request lifecycle: showing a loading state while data is being fetched, displaying the results in a responsive grid once they arrive, and handling errors gracefully if the request fails.

# 2. Objectives

- Connect the application to a free, public API using the Fetch API.
- Display a loading spinner while data is being downloaded.
- Render the fetched data in a responsive data grid.
- Reuse existing components (Buttons) within the new feature instead of duplicating code.
- Handle network/API errors gracefully.
- Deploy the updated application and push all changes to the existing GitHub repository.

# 3. Work Completed

- Selected the RandomUser API (https://randomuser.me) as the public data source — free, requires no API key, and returns multiple records suited to a grid layout.
- Built a new DataGrid component that fetches data using the Fetch API inside a useEffect hook, so data loads automatically when the component mounts.
- Implemented three distinct UI states: loading (animated spinner), error (friendly error message), and success (populated grid).
- Styled the loading spinner and data cards to match the existing neon theme, reusing the app's CSS custom properties (--neon, --neon-soft, --neon-border) so the new section automatically follows the selected color theme.
- Reused the existing Buttons component for a “Fetch New Data” action, avoiding writing a separate button from scratch.
- Added a new numbered section (“04 — Fetching Live Data”) to the main app layout, consistent with the structure of the existing sections.
- Committed and pushed all changes to the existing GitHub repository, then redeployed the live GitHub Pages site.
- Diagnosed and resolved a live-site update issue after deployment, confirmed to be a browser caching problem rather than a deployment failure, by checking the gh-pages branch commit history directly on GitHub.
- Refactored all components (Buttons, TextInput, ProfileCard, DataGrid, App) into more concise code with minimal inline comments, keeping only section-identifying comments.

# 4. Code Section — Implementation Explanation

## 4.1 Fetching Data with the Fetch API

The DataGrid component uses three pieces of state — the fetched users, a loading flag, and an error message — to represent the full lifecycle of a network request. The fetch itself runs inside a function that can be called again later (for the refresh button), and once on initial mount via useEffect.

```
const [users, setUsers] = useState([]);
const [loading, setLoading] = useState(true);
const [error, setError] = useState(null);

function fetchUsers() {
  setLoading(true);
  setError(null);

  fetch("https://randomuser.me/api/?results=6")
    .then((res) => (res.ok ? res.json() : Promise.reject("Network error")))
    .then((data) => {
      setUsers(data.results);
      setLoading(false);
    })
    .catch((err) => {
      setError(err.toString());
      setLoading(false);
    });
}

useEffect(() => {
  fetchUsers();
}, []);
```

## 4.2 Loading Spinner

While loading is true, a spinner and “Loading data...” message are shown instead of the grid. The spinner itself is a plain CSS-animated circle that inherits the app's current theme color through the --neon CSS variable.

```
{loading && (
  <div className="loading-spinner-wrapper">
    <div className="loading-spinner"></div>
    <p>Loading data...</p>
  </div>
)}
```

```
.loading-spinner {
  width: 46px; height: 46px; border-radius: 50%;
  border: 3px solid rgba(255,255,255,0.1);
  border-top-color: var(--neon);
  animation: spin 0.8s linear infinite;
}
@keyframes spin { to { transform: rotate(360deg); } }
```

## 4.3 Rendering the Data Grid

Once data has successfully loaded, the users array is mapped into a responsive CSS grid of styled cards, each showing the fetched person's photo, name, email, and location.

```
{!loading && !error && (
  <div className="data-grid">
    {users.map((u) => (
      <div className="data-card" key={u.login.uuid}>
        <img src={u.picture.medium} alt={u.name.first} className="data-card-img" />
        <h4>{u.name.first} {u.name.last}</h4>
        <p className="data-card-email">{u.email}</p>
        <p className="data-card-location">{u.location.city}, {u.location.country}</p>
      </div>
    ))}
  </div>
)}
```

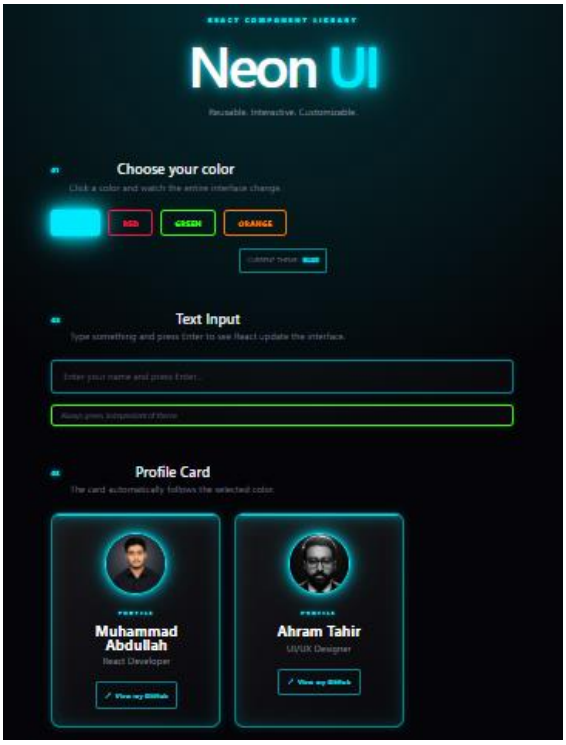
### 4.4 Reusing the Buttons Component

Rather than building a new button just for the refresh action, the existing Buttons component from Week 1 was reused directly, passing the current theme as its color prop so it automatically matches the selected theme.

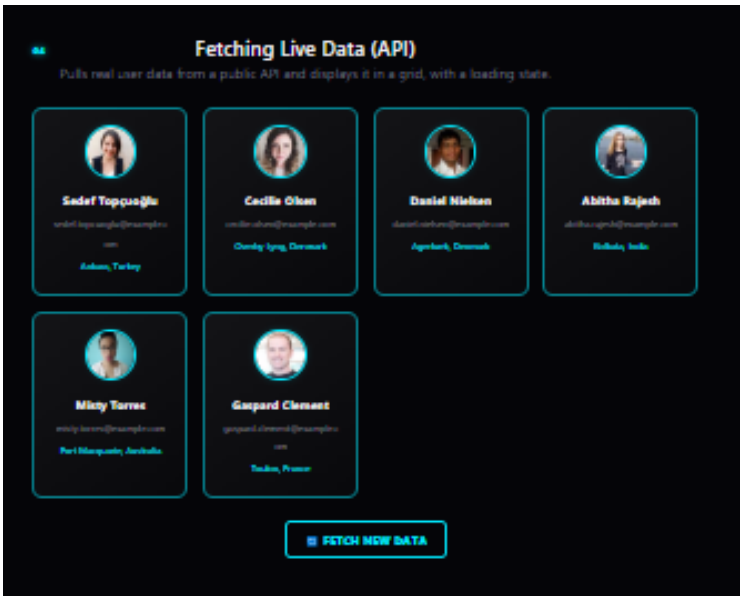
```
<Buttons
  text="🔄 FETCH NEW DATA"
  color={theme}
  size="medium"
  onClick={fetchUsers}
/>
```

## 5. Screenshots

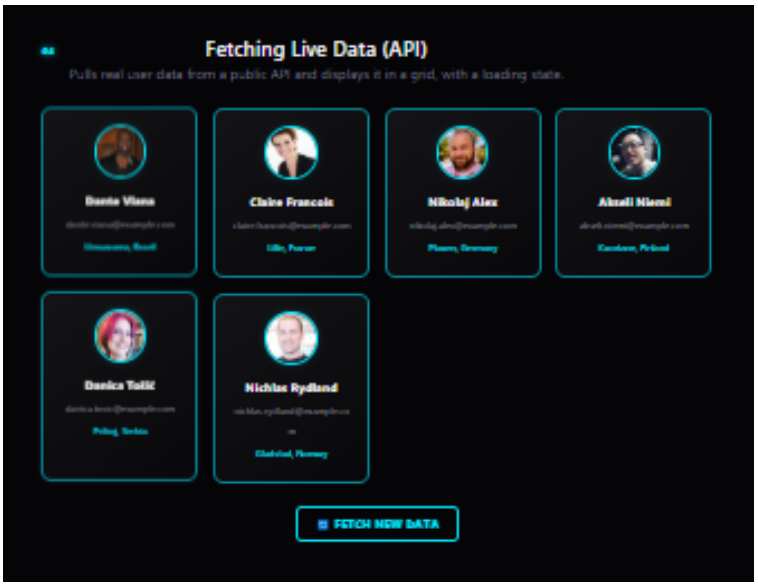
Application running:



Application running — data grid populated with results:




Refresh action in use:




## 6. Challenges Faced

- Selecting an appropriate public API that returned multiple records suited to a grid layout, rather than a single value.
- After deploying the update, the live GitHub Pages site initially appeared unchanged; this was traced to browser caching rather than a failed deployment, confirmed by checking the gh-pages branch commit timestamp directly on GitHub.
- Handling and displaying network/API errors gracefully without breaking the rest of the page.

## 7. Learnings

- Learned how to use the Fetch API with promises to retrieve data from a public endpoint.
- Learned how to represent and manage the three key states of an asynchronous request: loading, success, and error.
- Learned how the useEffect hook is used to trigger a data fetch when a component mounts.
- Learned how GitHub Pages deployment (gh-pages branch) is separate from the main branch, and how to verify a deployment directly through commit history rather than relying on a possibly-cached browser view.

## 8. Conclusion

By the end of Week 2, the Neon UI project was successfully extended to fetch and display live data from a public API, complete with a loading state, error handling, and a themed, responsive data grid. The implementation reused existing components from Week 1 wherever possible, keeping the codebase consistent and avoiding duplicated logic. The updated project was deployed and verified as live on GitHub Pages.

## 9. Project Link

GitHub Repository: https://github.com/muhammadabdullah023-hash/Reusable-UI